

Final report : Earthquake Early Prediction

As part of the course IFT3710/6759-H24

Done by Kheddache Yasmina (yasmina.kheddache@umontreal.ca), Philippe Schoeb (philippe.schoeb@umontreal.ca), Yannick Boily (yannick.boily@umontreal.ca), Mathieu D'Amours (mathieu.damours.2@umontreal.ca) and Wendtoin Elisée Kaboré (wendtoin.elisee.kabore@umontreal.ca).

26 avril 2024

1 Summary

To achieve our goal of implementing state of the art machine learning models on earthquake detection using the INSTANCE dataset [1], we have implemented an encoder/decoder based on the EQTransformer [2] model and two CNN models based on the ConvNetQuake [3] model. The transformers respectively reached an F1-Score of 0.98 and 1.0 on their respective test set. The best CNN we trained reached an F1-Score of 0.95 and ConvNetQuake's was 0.97. Then, we also built a metadata model since there is not many studies on it and it could be a novel contribution. The random forest we opted for hit a F1-Score of 0.93. Considering the much smaller training data used for our three models and the modest computational power needed for their training in comparison to the literature, we are satisfied with our models' performances. In addition, it demonstrates that even with limited resources, earthquake detection is a problem that machine learning tackles efficiently.

2 Introduction

Earthquakes are complex events that are hard to predict considering the variability of underlying geophysical processes. Originally, seisms' prediction was done using seismological models which analyse historical data, regional geology and tectonic plates' state. Nevertheless, these methods are rarely accurate when it comes to precisely predicting the moment, the place and the intensity of earthquakes. This past decade, the advancements of machine learning and the recent improvements of earthquakes' data has opened a path for deep learning to explore earthquakes' prediction and detection. With this in mind and considering the severe consequences earthquakes have in many regions, our team has decided to work on earthquake's prediction with the ultimate goal of preventing some losses during these catastrophes. However, after seeing the databases we had access to and after reading papers on the subject, we rapidly realised our data was not fit for our goal of predicting earthquakes ahead of time since the first part of signals before the event was basically pure noise. It was fit for a task of earthquake detection. That is being able to distinguish an earthquake from noise or being able to detect its start and its end. So, we made the choice to change our objective for seism detection rather than prediction so we could keep working with our databases (hence the misleading title).

Concretely, our objectives for this project was to recreate the best deep learning algorithms on a different dataset than their original's and to create a model that utilizes metadata for earthquake detection.

To achieve our goals, we have conceived a random forest classifier for the metadata, two CNN models for signal classification and an encoder/decoder for signal classification in addition to phase picking.

3 Literature Review

During our research on the subject, we found plenty of papers featuring promising results, such as ConvNetQuake [3] and EQTransformer [2]. Part of our work was greatly inspired by these two papers. ConvNetQuake utilized a convolutional neural network for detecting earthquakes and their location. This model simply uses 8 convolutional layers followed by a flattening and a fully connected layer. It was trained and tested on earthquake and noise waveforms originating from Oklahoma, in the United States. It reached a detection precision of 94.8% and a recall of 100%, without considering location errors. As for EQTransformer, it consists in detecting simultaneously an earthquake and its phases (P-phase and S-phase) using a deep-learning model with an encoder and three decoders, one for each task. More details about this architecture will be given in the *Methodology* section, since we implemented it. Concerning the two phases, the P-phase is the primary wave of an earthquake and it is usually followed by a stronger secondary wave, the S-phase. EQTransformer was previously used on the STEAD dataset [4] and it showed outstanding results. The model was trained on 85% of the data, which contained 1M earthquake and 300K noise waveforms and only got perfect scores on the test set for earthquake detection. Indeed, it had a precision, a recall and an F1 score of 1.

All in all, we have not found any study utilizing the metadata that is a part of several databases. In addition, the only benchmark we found revolving around earthquake detection for the INSTANCE dataset [1] was done using the first 10 seconds of the 120 seconds recording. It comes from ML4Quake, a project done for the Morocco Solidarity Hackathon [5]. Using a random forest, they achieved 87% accuracy and an F1 score of 84% for earthquake detection.

4 Dataset

For our project, we have decided to work with INSTANCE, the Italian seismic dataset for machine learning [1]. We made this choice because of the size and the quality of the dataset. Indeed, it is composed of nearly 1.2 million three-component waveform traces coming from more than 54 000 earthquakes of magnitude between 0.0 and 6.5, 132 330 3-channel noise waveform traces and 115 metadata for each event (earthquake waveform) providing information on the recording stations in Italy, the earthquake source, the trace features and quality. Every waveform is a 120 seconds recording of ground motion in three dimensions, sampled at 100 Hz. Thus, every waveform is a 12 000 by 3 matrix. Each noise waveform also has corresponding metadata, but there are fewer than event's. These data points consist of information about the location of the base stations generating the signals and other metadata about the signals.

In our experiments, we considered and tried enhancing our data by adding gaussian noise at varying SNR (signal

to noise ratio) to augment the dataset. We then computed new metadata for these augmented signals to have more metadata. However, after trying this, we encountered some limitation caused by the size of the database(180GB) and the restriction on the cluster. We also noticed that adding noise to the signal increased the size of the data in the hdf5 file as adding variation in the signal was reducing compression in the file. As we were already not able to use the full dataset in the cluster we decided to not create new noise, metadata or add gaussian noise to our signal data.

5 Methodology

In our work, we employed a comparative approach by evaluating various models and comparing their performances. We also used an iterative approach to fine-tune our models while exploring different features and hyperparameters. Visualizing the results facilitated easier interpretation of the model performances. We conducted several experiments with different model types, starting with Metadata Modeling – Random Forest and CNN models, and eventually using EQTransformer. Starting with preliminary tests, we chose to assess the effectiveness of CNNs on instance databases for seismic signal detection because CNN networks are commonly used for their ability to capture spatial dependencies in the data and applied in detection-related tasks such as estimating earthquake magnitude and location.

5.1 Preliminary tests

To acquaint ourselves with h5py files for reading and utilizing earthquake signals, we conducted tests using 1000 samples of noise and 10,000 samples of earthquake signals from the instance Database. We extracted earthquake and noise signals and respectively assigned target labels of 1 and 0. In addition, we used CSV files that contain information about the earthquake and noise signal databases. The following steps were taken:

- Loading Data from h5py files, then extracting information about the databases of noise and earthquake signals, such as trace names and source type, from CSV files.
- Combining earthquake and noise signals, then splitting them into training, validation, and testing sets.
- Training a Convolutional Neural Network (CNN) model using the training data. We used the complete signals without performing any prior preprocessing.
- Evaluating the performance of the trained model using the evaluation and testing datasets.

The results indicated rapid growth followed by stabilization of accuracy and a decrease in loss. Thus, the model began to efficiently learn the features of the training data, and the accuracy stabilized as it reached its performance limits on the training set and its test value of 0.91.

5.2 Metadata Modelling - Random Forest

We now present the model trained on the metadata extracted. This model uses some data derived from the signals and other type of information, i.e., the base stations geolocation. This data is tabular and comes from the CSV files of the INSTANCE sample [1].

First, the data has been enriched by adding the cross-feature *station_grid_cell* which is the combination of the latitude and longitude locations of the base stations. It was then one-hot encoded while the rest of the metadata features were scaled between 0 and 1. Next, a basic Random Forest model was trained using the *scikit-learn* package. Its hyperparameters have also been tuned using a scikit package called *scikit-optimize*. This looks for the best hyperparameters in a space that we have defined for these: *n_estimators*, *max_depth*, *min_samples_leaf*, *max_leaf_nodes*.

The experiments were logged and visualized using MLflow. We have trained the Random Forest classifier using either balanced or unbalanced metadata and then evaluated it on a validation set to get the best hyperparameters. After that, we have evaluated on a test set the best model.

The best results were found with a balanced training and test sets and here they are:

$$TP = 97, FP = 3, TN = 101, FN = 7$$

5.3 Signals Modelling - CNN

We chose Convolutional Neural Networks (CNNs) for earthquake detection because they are commonly used for their ability to capture spatial dependencies in the data. In the first step, the dataset used for the experimentation comprises 2000 signals, encompassing both earthquake and noise data, each characterized by three channels. We employed TensorFlow's Keras module for model development. Before training, we conducted a random split of the dataset, allocating 20% for testing and another 20% for validation. Our experimentation involved the implementation of two distinct CNN models. One model was designed with reduced complexity, prioritizing expedited training time and enhanced generalization. The architectures of these two CNN models are depicted in Figure 1. CNN model1 consists of 24,572,353 parameters (93.74 MB), while CNN model2 comprises 519,681 parameters (1.98 MB). During the training phase, we used Adam optimizer. Hyperparameters were set as follows: Epochs = 150, Batch-size = 64. The input shape was defined as (3, 12000), reflecting the three channels and a time series of length 12000. Both models underwent the same training process that encompassed multiple epochs. Upon completion of training, we evaluated the models' performance using appropriate metrics, such as accuracy, precision, recall, and F1-score, on the validation set. Additionally, we assessed their generalization capabilities by evaluating performance on the separate testing set. To investigate the real efficiency of our best CNN model in detecting earthquake signals, the second step of our work was to retrain and test the CNN on a large database of 22410/2800/2800 (training/validation/test).

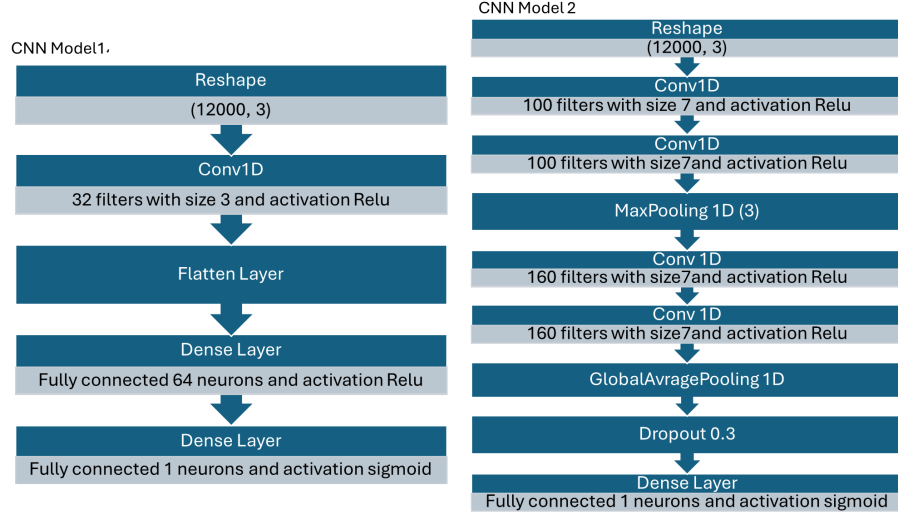


Figure 1: CNN Models architecture

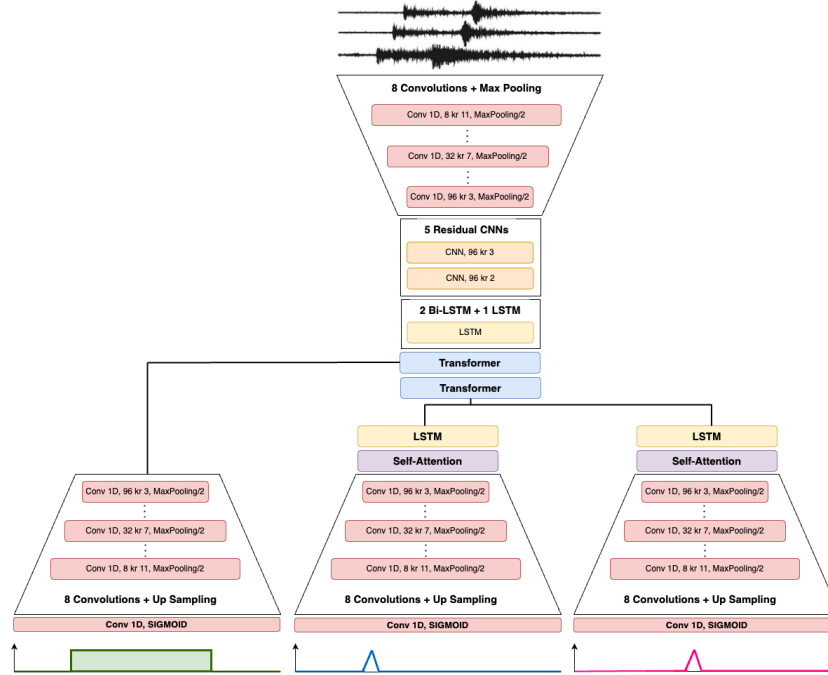


Figure 2: EQ Model architecture

5.4 Signals modelling - EQModel

This method uses the EQTransformer [2] encoder/decoder architecture and adapts its layers and concepts to work with the INSTANCE dataset [1]. The network has a multi-task objective of detecting the earthquake and the samples that represent the P-Phase and S-Phase. The idea behind is that learning to pick the P-Phase and S-Phase will help the model better encode and detect the earthquake itself. Hence the model has a deep encoder composed of convolutions,

RNNs, transformers and 3 deep decoders, one for each task. The model relies on the convolution layers to extract the useful features in such a way that the only preprocessing done on the data is a simple normalization. The model uses the transformers for their attention capability to get good latent representation before starting the decoding and the RNNs sets the positional encoding needed by the transformers. Each decoder outputs a sequence of probability, each probability corresponding to each sample of one signal input. The output of the detection decoder has all its probabilities representing the earthquake portion of the signal set to 1 and the rest set to 0 for noise: that gives a square box shape target. For phase picking, a triangular shape is instead used and centered on the sample of interest. See figure 2 for more details. In the original paper, they tried Gaussian and other shape target but they were less successful. To properly predict, we use a height threshold to detect the peaks and box shape. We ended up with a deep network of 650k trainable parameters. The data has been split into 22410 training, 2800 validation and 2800 test data points evenly composed of noise and earthquake signals. Having a balanced dataset made it easier to interpret the quality of the model. We used the validation loss to pick the best model. The optimizer used is Adam with a scheduler starting at 0.001 and dividing the learning rate by 10 after 20 epochs. Also we set up the training to stop either after 12 consecutive epochs without improvement or after 30 epochs.

6 Results

We compare the results obtained in this study against the one presented in the reference papers. We admit that the comparison might be somehow biased as we did not test on the same dataset.

Model	Precision	Recall	F1-Score	Training size	Test size	Location
EQModel [This study]	0.98	0.98	0.98	22K	2K	Italy
CNN 1 [This study]	0.95	0.95	0.95	22K	2K	Italy
Random forest	0.94	0.93	0.93	1.6K	214	Italy
EQTransformer [2]	1.0	1.0	1.0	1.2M	113K	Global
ConvNetQuake [3]	0.95	1.0	0.97	131K	703K	US
CPIC [6]	0.97	0.98	0.97	48K	12K	China

Table 1: Metric comparison

6.1 EQModel

Our EQModel achieves an accuracy and f1 score greater than 98%. It produces 8 false positive and 35 false negative out of 5600 test signals. It uses less training size than CPIC and ConvNetQuake but gives comparable if not better results in terms of classification. The only model that clearly surpasses it in all metrics is the original EQTransformers

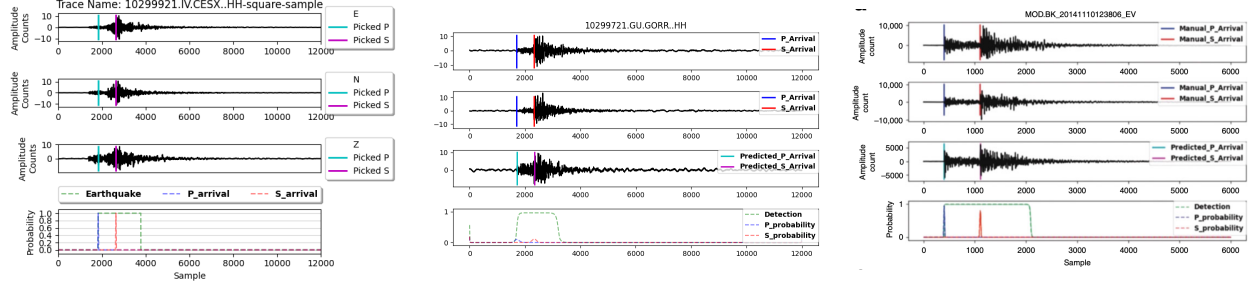


Figure 3: From left to right, signal input and output: Training sample, EQModel prediction, Original EQTransformer prediction

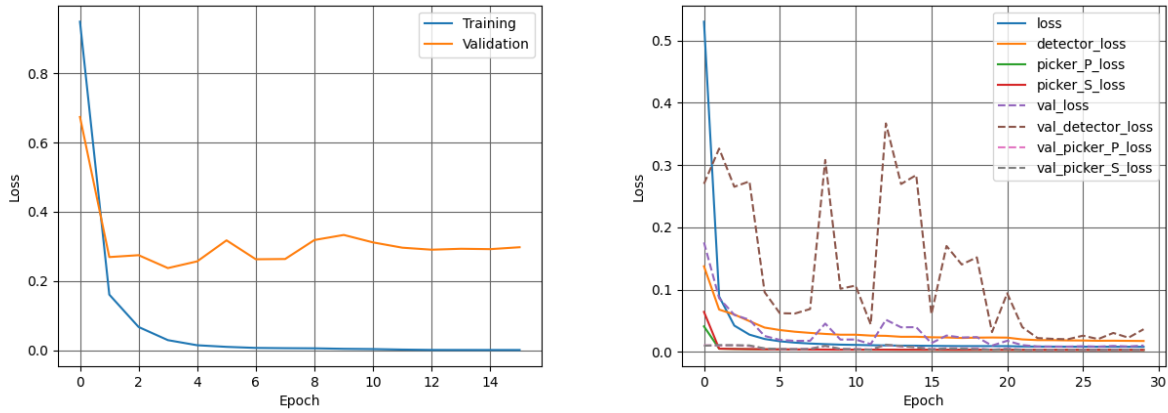


Figure 4: Left: CNN loss, Right: EQModel loss

which has been trained on 50x more data for O(89) hours on GPUs. EQModel has been trained for O(5) hours on a simple Apple M1 chip. It was faster training the model on the cluster with GPU, which is about half the time but the hyper-parameters used for training on the cluster did not result in better models. The EQModel results however are very close to the original EQTransformer model with very less resources.

Looking closely at the EQModel training history 4, we observed that the detection validation loss was very unstable for the first 20 epochs. Actually the learning rate scheduler explained in the methodology above was very helpful to lower and stabilize the validation loss.

Also, as it is a multi-task objective, we had to fine-tune and weight the different outputs losses. The best model has been achieved using a weight of 0.1 for detection and 0.45 for each of picking phase loss. In the discussion section we'll get back to it in more details.

6.2 CNN models

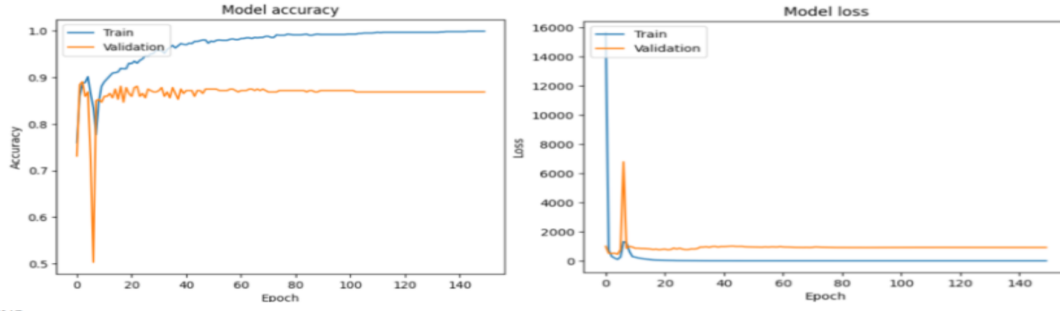
The model CNN1 demonstrates good performance across all metrics, The overall accuracy is 86%, suggesting that it effectively discriminates between the two classes. Based on classification reports, the results indicate that precision, recall, and F1cores scores for CNN1 are high for both classes which suggests a good balance between precision and recall. CNN2 classifier performs relatively well in identifying instances of class 0 but struggles more with class 1. The precision and recall scores for class 1 are imbalanced. The test accuracy of CNN2 is 75%. The F1-scores are 78%, 72% respectively for class 0 and 1. This score demonstrates good harmony between precision and recall for both classes. The results shown in Figure 5 for CNN 1 indicate a stabilization of the loss and accuracy curves, indicating that the models converge and do not appear to overfit the data. The peaks in the loss and accuracy curves observed at the start of training are attributed to the initial training of the model (adjusting its weights). The model converges and the fluctuations decrease and the model reaches its maximum performance. For CNN2, we noticed a similar evolution of the loss and precision curves, effectively generalizing to new data while maintaining a consistent trajectory despite its non-smoothness. In conclusion, the first model CNN1 outperforms the second one in all metrics, demonstrating higher precision, recall, F1-score, and accuracy. The second classifier, while achieving a decent accuracy, shows imbalanced performance between the two classes.

The obtained test results for training CNN1 with a larger dataset extracted from the INSTANCE database, containing 22410/2800/2800 for training/validation/test are presented in Table 1. The overall accuracy for test is 95% suggesting that there is an improvement in recognising both earthquake signals and noise. The obtained loss for training and validation is presented in Figure 4. The model reaches rapidly its maximum performance without overfitting.

7 Discussion

The EQTransformer has almost 100% in all metrics so we knew that it would be difficult to beat. Our goal was to not only reproduce similar result on a different dataset but also use very less resources at the point it could be trained on a regular machine. Inspired from the EQTransformer architecture, EQModel achieves that goal. We enlarged the encoder and decoder layer by adding new convolutions and we used 1/50 of the original data size. One of the main problem we faced was related to the phase picking learning. Phase P/S decoders loss quickly goes to zero. Actually their true target is a sequence of 12000 probabilities where 11960 are zeros. A network that does not learn and outputs all zeros already produces a very small binary cross entropy loss. In addition, the detection task loss was more than 10x bigger hence undermining phase decoders update signals. That has been exacerbated compared to the original EQTransformer as our data input and model output size are 2x larger. To tackle that issue, we increased the weight of phase decoders loss and reduced the on for detection task. Also, we had to largely reduce the threshold height to detect pick in order to properly predict. See EQModel prediction example [3] which shows a target sample, a prediction of our EQModel and a prediction of the original EQTransformers. We think that making larger the base of the triangle

CNN1



CNN2

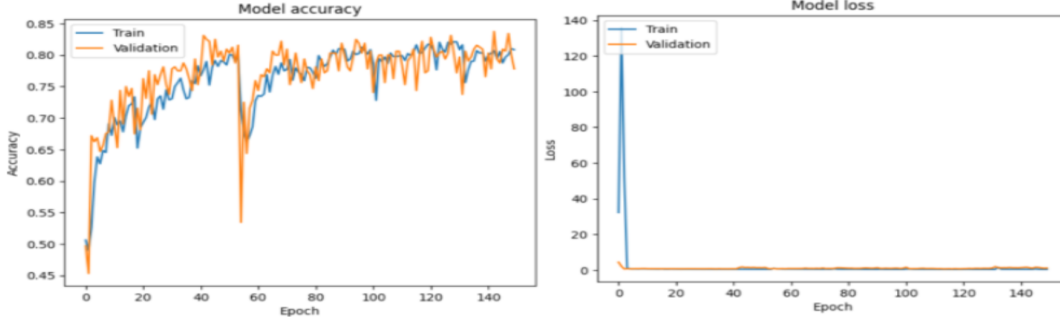


Figure 5: Loss and Accuracy for CNN1 and CNN2 models

shape, fine-tuning more the loss weights and using more data would help improve the phase picking learning.

Using more data would overall improve as well the detection accuracy. Even the training detection accuracy quickly goes to zero. That signals that EQModel has a lot of capacity which will benefit from having more data. Same thing is noticed on the CNN 1 loss where over-fitting starts after the 5th epoch. Despite the scheduler that reduces the CNN 1 learning rate over time, the validation loss does not improve and the training stops after 12 consecutive epochs without improvements. We should enforce more regularization on both EQModel and CNN 1 (even reduce CNN 1 parameters) to fine-tune as it will lead to better generalization.

The capacity of EQModel comes from the fact that it uses a deep network but as well that it uses RNNs and transformers blocks. This combination is what set EQModel apart from CPIC, ConvNetQuake and other previous state of the art models. Their incorporation effectively helps the model take a larger input size and produce a larger multitask output while still achieving state of the art results.

On the other note, we have not found any model utilizing metadata in the literature so we cannot compare it to a similar model. However, as seen in table 1, this model has had worst performances than most of the models utilizing signal data. Considering the size of its training data was much smaller than the other models', its capacities have potential. Moreover, the combination of a signal model and this metadata model could be a subject of interest for more complex tasks. The obtained results using CNN model reach our expectation and are comparable to different results obtained in [7] and the [8].

8 Conclusion

To conclude, we have accomplished our purpose for this project by implementing three major models and training them on much fewer data, using modest sources of computational power. Indeed, the results of our two signal models come close to the literature while using considerably less resources, thus indicating training for earthquake detection without state of the art resources is still successful and useful. Furthermore, our metadata model also demonstrates potential in addition to being easily interpretable. For future work, uniting a signal model to a metadata model could be interesting as well as incorporating optimization of hyperparameters for signal models too. It would also be interesting to approach to our initial goal of earthquake prediction by adapting and training our models on only the first few seconds of each signal to see how it affects their performances.

9 Data and Code availability

Download either samples or full data from INSTANCE github [1] and reproduce/extend our experiments from our github: <https://github.com/damoursm/earthquake> .

10 Contributions

Yasmina Kheddache:

- Preliminary tests realization
- Participation in the Establishment of the literature review and the followed methodology in the project
- Implementation and testing of the two CNN models
- Participation in drafting homework documents and the final report

Yannick Boily:

- Participation in the Establishment of the literature review and the followed methodology in the project
- Implementation of some utility code and test on the database
- Participation in drafting homework documents and the final

Wendtoin Elisée Kaboré:

- Created the scripts and generic templates to automate/make it easy to train on cluster including testing and finding the optimal configurations
- Created lazy loading dataset to allow training on big datasets without loading everything in memory
- Adapted and trained INSTANCE on the EQTransformer architecture style

- Participated in drafting homework documents and final report

Philippe Schoeb:

- Participation in the establishment of the literature review and the followed methodology in the project
- Tested EQModel and cluster usage
- Participated in drafting homework documents and final report

Mathieu D'Amours:

- Metadata features engineering (features_engineering.py)
- Model training code for Random Forest (training.py)
- Hyperparameters tuning optimizer (main.py)
- MLflow experiments tracker (main.py)
- Participated in drafting homework documents and final report

References

- [1] A. Michelini *et al.*, *Instance – the italian seismic dataset for machine learning*, Earth System Science Data **13** (2021) .
- [2] W. Z. L. Y. C. . G. C. B. S. Mostafa Mousavi, William L. Ellsworth, *Earthquake transformer—an attentive deep-learning model for simultaneous earthquake detection and phase picking*, Nature Communications (2020) .
- [3] M. D. T. Perol, M. Gharbi, *Convolutional neural network for earthquakedetection and location*, Sci. Adv.4, e1700578 (2018) .
- [4] S. Y. Z. W. . B. G. C. Mousavi, S. M., *In stanford earthquake dataset (stead)*, A Global Data Set of Seismic Signals for AI (2019) .
- [5] M. S. Hackathon, *Leveraging ai for natural disaster management : Takeaways from the moroccan earthquake*, 2023.
- [6] L. Zhu *et al.*, *Deep learning for seismic phase detection and picking in the aftershock zone of 2008 mw7.9 wenchuan earthquake*, Physics of the Earth and Planetary Interiors **293** (2019) 106261.
- [7] M. A. Bilal, Y. Ji, Y. Wang, M. P. Akhter and M. Yaqub, *An early warning system for earthquake prediction from seismic data using batch normalized graph convolutional neural network with attention mechanism (bngcnnatt)*, Sensors (Basel) **22** (2022) 6482.
- [8] T. Tokuda and H. Nagao, *Seismic-phase detection using multiple deep learning models for global and local rep-*

resentations of waveforms, Geophysical Journal International **235** (2023) 1163.